

1 Task 1

You are developing a simple library management system.

Create a class `Book` with:

- **Private fields:** `title` (String), `author` (String), `yearPublished` (int)
- A **default constructor** that sets "Unknown Title", "Unknown Author", 0
- A **parameterized constructor** that accepts all three values
- A method `displayInfo()` that prints the book details in this format:
Title: [title] | Author: [author] | Published: [year]
- Use encapsulation (private fields + public getters)

In the `main` method,

Create a 2D jagged array of `Character` objects with 2 rows and display their information.

- Row 0 has 5 elements
- Row 1 has 2 elements

2 Task 2

A company wants to track how many employees are currently working.

Create a class `Employee` with:

- Instance fields: `name` (String), `id` (int)
- Static field: `totalEmployees` (int) initialized to 0
- A parameterized constructor that takes name and id, increments `totalEmployees`, and uses `this` keyword
- A method `showDetails()` that prints: "ID: [id] | Name: [name]"
- A static method `getTotalEmployees()` that returns the total count

In `main()`, create 3 employees and print each employee's details + total number of employees at the end.

3 Task 3

You are modeling different types of vehicles for a transportation app.

Create a base class **Vehicle** with:

- Fields: **brand** (String), **speed** (int)
- Constructor to initialize them
- Method **move()** that prints "[brand] is moving at [speed] km/h"

Create a subclass **Car** that extends **Vehicle** and adds:

- Extra field: **fuelType** (String)
- Override **move()** to print: "[brand] car is driving at [speed] km/h using [fuelType]"
- Additional method **honk()** that prints "Beep Beep!"

In **main()**, create one **Vehicle** and one **Car**, then demonstrate method overriding by calling **move()** on both objects (use superclass reference for **Car**).

4 Task 4

Design a hierarchy for electronic devices.

Create classes with multilevel inheritance: **Device** → **Electronics** → **SmartPhone**

- **Device**: field **powerConsumption** (double), method **turnOn()** → "Device is turning on"
- **Electronics**: field **warrantyYears** (int), constructor using **super()**, method **showWarranty()**
- **SmartPhone**: field **osVersion** (String), override **turnOn()** using **super.turnOn()** + add "SmartPhone is booting [osVersion]"

In **main()**, create a **SmartPhone** object and call **turnOn()** and **showWarranty()**.

5 Task 5

A drawing application needs to handle different shapes.

Create a base class **Shape** with:

- **Abstract method** **draw()** (make **Shape** abstract)
- Concrete method **getType()** that returns "Generic Shape"

Create three subclasses (hierarchical):

- **Circle** → override **draw()** → "Drawing a circle"

-
- `Rectangle` → override `draw()` → "Drawing a rectangle"
 - `Triangle` → override `draw()` → "Drawing a triangle"

In `main()`:

- Create an array of `Shape` references (size 4)
- Put different shape objects into the array (polymorphism)
- Loop through the array and call `draw()` on each (demonstrate runtime polymorphism)